$\forall$ NFA $\exists$ DFA w/ same L but w/ $Q_{DFA} = 2^k Q_{NFA}$

- A major reason to use NFA even on regular languages.

Now $\delta(q, x) = \{q_1', q_2', \ldots\}$ & $\lambda$ gaps allowed.

# L08: Regular Expressions

Regular Expressions describe languages

E.g. $(a+b\cdot c)^*$ describes $\{a,bc\}^* = \{\lambda, a, bc, abc, bca, \ldots\}$

Primitive regular expressions: $\emptyset$, $\lambda$, $a$
(empty set; empty input; letter)

Given regular expressions $r_1, r_2$ have operations:
- $L(r_1+r_2) = L(r_1) \cup L(r_2)$ union
- $L(r_1 \cdot r_2) = L(r_1)L(r_2)$ concatenation
- $L(r_1^*) = (L(r_1))^*$ repeating $\cup\ \lambda$

E.g. $r = (aa)^*(bb)^*b \vdash L(r) = \{a^{2n}b^{2m}b \mid n,m \geq 0\}$

Languages generated by regexes $\equiv$ regular languages

$\therefore$ DFAs, NFAs, Regexes accept regular languages

Consider pattern $p = p_1 p_2 \cdots p_n$ $p_i \in \Sigma$ & text $t = t_1, t_2, \ldots, t_m$ $t_k \in \Sigma$

Want to match $p$ in $t$ in $O(n+m)$ via KMP algorithm

First construct pattern matching machine $M_p$ using DFA:

$p_1$ → 0 $\xrightarrow{p_1}$ 1 $\xrightarrow{p_2}$ 2 ⋯ $\xrightarrow{p_{n-1}}$ $n-1$ $\xrightarrow{p_n}$ (n)

State $i$ is pointer to $i$th position in $p$

KMP $M_p$ Construction

Then extend the DFA where state $j$ corresponds to prefix of $p$: $p_1, p_2, \ldots, p_j$. $M_p$ starts at state 0 w/ input $t$

1. IF $t_1 = p_1$: $M_p$ enters state 1, reads $t_2$
2. ELSE: $M_p$ stays in state 0, reads $t_2$
3. IF $t_{k+1} = p_{j+1}$: $M_p$ enters state $j+1$, reads $t_{k+2}$
4. ELSE: $M_p$ enters highest state $i$ s.t. $p_{1:i}$ is a suffix of $t_{1:k+1}$
5. $i$ obtained from Failure Fxn Algorithm; $f(j)$ is largest $s < i$ for which $p_{1:s}$ is suffix of $p_{1:j}$